

[Features](#)[Enterprise](#)[Pricing](#)[Resources](#)[Sign in](#)[Download](#)[Blog / Research](#)

Scaling long-running autonomous coding

Jan 14, 2026 by Wilson Lin

Table of Contents

[The limits of a single agent](#)[Learning to coordinate](#)[Planners and workers](#)[Running for weeks](#)[What we've learned](#)[What's next](#)

We've been experimenting with running coding agents autonomously for weeks.

Our goal is to understand how far we can push the frontier of agentic coding for projects that typically take human teams months to complete.

This post describes what we've learned from running hundreds of concurrent agents on a single project, coordinating

their work, and watching them write over a million lines of code and trillions of tokens.

The limits of a single agent

Today's agents work well for focused tasks, but are slow for complex projects. The natural next step is to run multiple agents in parallel, but figuring out how to coordinate them is challenging.

Our first instinct was that planning ahead would be too rigid. The path through a large project is ambiguous, and the right division of work isn't obvious at the start. We began with dynamic coordination, where agents decide what to do based on what others are currently doing.

Learning to coordinate

Our initial approach gave agents equal status and let them self-coordinate through a shared file. Each agent would check what others were doing, claim a task, and update its status. To prevent two agents from grabbing the same task, we used a locking mechanism.

This failed in interesting ways:

1. Agents would hold locks for too long, or forget to release them entirely. Even when locking worked correctly, it became a bottleneck. Twenty agents would slow down to the effective throughput of two or

- three, with most time spent waiting.
2. The system was brittle: agents could fail while holding locks, try to acquire locks they already held, or update the coordination file without acquiring the lock at all.

We tried replacing locks with optimistic concurrency control. Agents could read state freely, but writes would fail if the state had changed since they last read it. This was simpler and more robust, but there were still deeper problems.

With no hierarchy, agents became risk-averse. They avoided difficult tasks and made small, safe changes instead. No agent took responsibility for hard problems or end-to-end implementation. This led to work churning for long periods of time without progress.

Planners and workers

Our next approach was to separate roles. Instead of a flat structure where every agent does everything, we created a pipeline with distinct responsibilities.

- **Planners** continuously explore the codebase and create tasks. They can spawn sub-planners for specific areas, making planning itself parallel and recursive.
- **Workers** pick up tasks and focus entirely on completing them. They don't coordinate with other workers

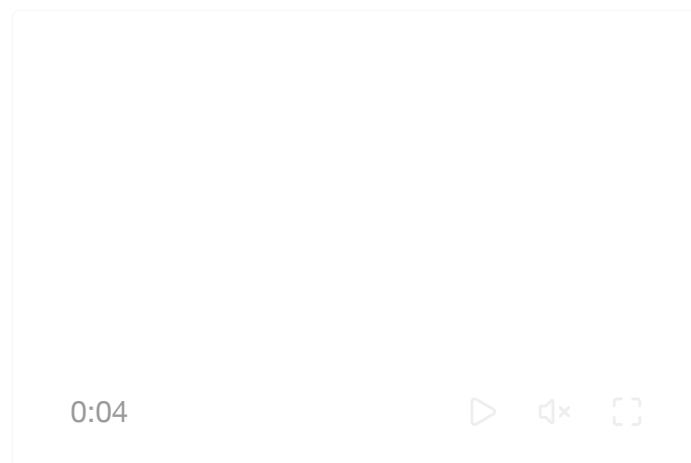
or worry about the big picture. They just grind on their assigned task until it's done, then push their changes.

At the end of each cycle, a judge agent determined whether to continue, then the next iteration would start fresh. This solved most of our coordination problems and let us scale to very large projects without any single agent getting tunnel vision.

Running for weeks

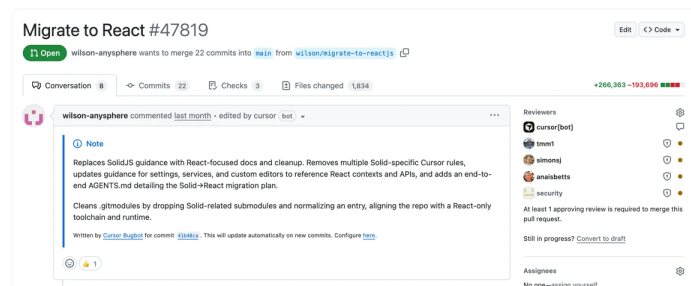
To test this system, we pointed it at an ambitious goal: building a web browser from scratch. The agents ran for close to a week, writing over 1 million lines of code across 1,000 files. You can explore the [source code on GitHub](#).

Despite the codebase size, new agents can still understand it and make meaningful progress. Hundreds of workers run concurrently, pushing to the same branch with minimal conflicts.



While it might seem like a simple screenshot, building a browser from scratch is extremely difficult.

Another experiment was doing an in-place migration of Solid to React in the Cursor codebase. It took over 3 weeks with +266K/-193K edits. As we've started to test the changes, we do believe it's possible to merge this change.



Another experiment was to improve an upcoming product. A long-running agent made video rendering 25x faster with an efficient Rust version. It also added support to zoom and pan smoothly with natural spring transitions and motion blurs, following the cursor. This code was merged and will be in production soon.

We have a few other interesting examples still running:

- Java LSP: 7.4K commits, 550K LoC
- Windows 7 emulator: 14.6K commits, 1.2M LoC
- Excel: 12K commits, 1.6M LoC

What we've learned

We've deployed billions of tokens across these agents toward a single goal. The system isn't perfectly efficient, but it's far more effective than we expected.

Model choice matters for extremely long-running tasks. We found that GPT-5.2 models are much better at extended autonomous work: following instructions, keeping focus, avoiding drift, and implementing things precisely and completely.

Opus 4.5 tends to stop earlier and take shortcuts when convenient, yielding back control quickly. We also found that different models excel at different roles. GPT-5.2 is a better planner than GPT-5.1-codex, even though the latter is trained specifically for coding. We now use the model best suited for each role rather than one universal model.

Many of our improvements came from removing complexity rather than adding it. We initially built an integrator role for quality control and conflict resolution, but found it created more bottlenecks than it solved. Workers were already capable of handling conflicts themselves.

The best system is often simpler than you'd expect. We initially tried to model systems from distributed computing and organizational design. However, not all of them work for agents.

The right amount of structure is somewhere

in the middle. Too little structure and agents conflict, duplicate work, and drift. Too much structure creates fragility.

A surprising amount of the system's behavior comes down to how we prompt the agents. Getting them to coordinate well, avoid pathological behaviors, and maintain focus over long periods required extensive experimentation. The harness and models matter, but the prompts matter more.

What's next

Multi-agent coordination remains a hard problem. Our current system works, but we're nowhere near optimal. Planners should wake up when their tasks complete to plan the next step. Agents occasionally run for far too long. We still need periodic fresh starts to combat drift and tunnel vision.

But the core question, can we scale autonomous coding by throwing more agents at a problem, has a more optimistic answer than we expected. Hundreds of agents can work together on a single codebase for weeks, making real progress on ambitious projects.

The techniques we're developing here will eventually inform Cursor's agent capabilities. If you're interested in working on the hardest problems in AI-assisted software development, we'd love to hear from you at hiring@cursor.com. 

Filed under: Research

Author: Wilson Lin

← Previous post
Best practices
for coding with
agents

Next post →
Building a better
Bugbot

Product	Resources	Company	Legal	Connect
Features	Download	Careers	Terms of Service	X
Enterprise	Changelog	Blog	Privacy Policy	LinkedIn
Web Agents	Docs	Community	Data Use	YouTube
Bugbot	Learn	Workshops	Security	
CLI	Forum	Students		
Pricing	Status	Brand		